

React Native with TypeScript for Beginners

This is a from-scratch introduction to React Native, using TypeScript from the start. You don't need prior TypeScript experience — we'll explain each concept as it comes up.

What You'll Need

- Node.js installed (v18 or newer)
- A code editor (VS Code recommended — it has excellent built-in TypeScript support)
- A phone with the **Expo Go** app, or a simulator/emulator on your computer

1. Create Your First App

```
npx create-expo-app@latest MyFirstApp
cd MyFirstApp
npx expo start
```

Expo's default template already uses TypeScript — you'll see `.tsx` files (not `.js`). A QR code appears in your terminal; scan it with Expo Go on your phone to see the app running live.

2. What Is TypeScript, Really?

JavaScript doesn't check what *type* of value a variable holds — you can put a number in a variable and then accidentally treat it like text, and JavaScript won't complain until something breaks at runtime.

TypeScript adds those checks **before** you run the app, right in your editor. It's the same JavaScript underneath, just with extra annotations that catch mistakes early.

```
let age = 25;
age = "twenty-five"; // TypeScript flags this
                        immediately, red squiggly line in your editor
```

That's really the whole idea: catch typos and mismatched data *while you type*, instead of discovering them when the app crashes on someone's phone.

3. Your First Typed Component

Open `App.tsx` :

```
import { StyleSheet, Text, View } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text>Hello, world!</Text>
    </View>
  );
}
```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

This looks almost identical to plain JavaScript React Native — and that's normal. TypeScript is additive; you'll notice it most once we start passing data between components.

4. Typing Your First Props

"Props" are how a parent component passes data to a child. In TypeScript, you describe the shape of those props with an `interface`:

```
import { View, Text, StyleSheet } from 'react-native';

interface GreetingProps {
  name: string;
}

function Greeting({ name }: GreetingProps) {
  return <Text>Hello, {name}!</Text>;
}

export default function App() {
  return (
    <View style={styles.container}>
      <Greeting name="Alex" />
    </View>
  );
}
```

```

    );
  }

  const styles = StyleSheet.create({
    container: { flex: 1, alignItems: 'center',
      justifyContent: 'center' },
  });

```

Try changing `<Greeting name="Alex" />` to `<Greeting name={42} />` — your editor will immediately underline it in red, because `42` is a number, not a `string`. That's TypeScript doing its job: catching the mistake before you even save the file.

Multiple props

```

interface GreetingProps {
  name: string;
  age: number;
}

function Greeting({ name, age }: GreetingProps) {
  return <Text>Hello, {name}! You are {age}
years old.</Text>;
}

// Used like:
<Greeting name="Alex" age={30} />

```

If you forget the `age` prop entirely, or pass a string instead of a number, TypeScript will tell you exactly what's wrong and where.

5. State: Making Things Interactive

`useState` remembers a value across re-renders — for example, a counter.

```
import { useState } from 'react';
import { View, Text, Pressable, StyleSheet }
from 'react-native';

export default function App() {
  const [count, setCount] = useState(0);

  return (
    <View style={styles.container}>
      <Text style={styles.count}>{count}</Text>
      <Pressable style={styles.button} onPress=
{() => setCount(count + 1)}>
        <Text style={styles.buttonText}>Add
One</Text>
      </Pressable>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, alignItems: 'center',
justifyContent: 'center' },
  count: { fontSize: 48, marginBottom: 20 },
  button: { backgroundColor: '#2f95dc',
paddingVertical: 12, paddingHorizontal: 24,
borderRadius: 8 },
  buttonText: { color: '#fff', fontSize: 16 },
});
```

Notice we didn't write `useState<number>(0)` anywhere — TypeScript is smart enough to look at the starting value (`0`)

and figure out on its own that `count` will always be a `number`. This is called **type inference**, and it means you often don't need to write types explicitly at all.

When you DO need to say the type

Inference isn't magic — if you start with an empty or ambiguous value, TypeScript can't guess what's coming later, so you tell it directly using `<AngleBrackets>`:

```
// Starting empty — TypeScript can't guess this
// will hold strings
const [name, setName] = useState<string>('');

// Starting as "nothing yet" — very common
// pattern
const [user, setUser] = useState<string | null>
(null);
```

`string | null` reads as "a string, OR nothing at all" — useful for things like "no user logged in yet."

6. Typing Text Input

```
import { useState } from 'react';
import { View, Text, TextInput, StyleSheet }
from 'react-native';

export default function App() {
  const [name, setName] = useState<string>('');

  return (
    <View style={styles.container}>
      <TextInput
```

```

        style={styles.input}
        placeholder="Enter your name"
        value={name}
        onChangeText={setName}
      />
      <Text style={styles.greeting}>
        {name ? `Hello, ${name}!` : 'Waiting for
your name...'}
      </Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, padding: 20,
justifyContent: 'center' },
  input: { borderWidth: 1, borderColor: '#ccc',
borderRadius: 8, padding: 10, marginBottom: 16
},
  greeting: { fontSize: 18 },
});

```

`onChangeText={setName}` works because `TextInput`'s `onChangeText` always hands you a `string` — the same type `setName` expects. TypeScript checks this behind the scenes automatically; you don't have to declare anything extra here.

7. Typing a List of Data

This is where TypeScript starts paying off noticeably. Say we have a list of todos — each one needs a consistent shape:

```

import { FlatList, Text, View, StyleSheet } from
'react-native';

// Describe the shape of one todo item
interface Todo {
  id: string;
  title: string;
  done: boolean;
}

const todos: Todo[] = [
  { id: '1', title: 'Learn TypeScript basics',
done: false },
  { id: '2', title: 'Build my first typed
component', done: true },
];

export default function App() {
  return (
    <View style={styles.container}>
      <FlatList
        data={todos}
        keyExtractor={({item}) => item.id}
        renderItem={({ item }) => (
          <View style={styles.item}>
            <Text>{item.title} – {item.done ?
'✅' : '⌚'}</Text>
          </View>
        )}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, paddingTop: 60,
paddingHorizontal: 20 },

```



```
    item: { padding: 15, borderBottomWidth: 1,
            borderBottomColor: '#eee' },
  });
```

Because `todos` is typed as `Todo[]` (an array of `Todo`), your editor now knows that inside `renderItem`, `item` definitely has `.id`, `.title`, and `.done` — and it'll autocomplete them for you. Try typing `item.` inside the `renderItem` function and watch the suggestions appear.

If you tried `item.titel` (a typo), TypeScript would immediately flag it instead of silently showing "undefined" on your screen at runtime — this is one of the most common beginner bugs TypeScript eliminates entirely.

8. Putting It Together: A Mini To-Do App

```
import { useState } from 'react';
import { View, Text, TextInput, Pressable,
  FlatList, StyleSheet } from 'react-native';

interface Todo {
  id: string;
  text: string;
}

export default function App() {
  const [task, setTask] = useState<string>('');
  const [todos, setTodos] = useState<Todo[]>
    ([]);

  const addTodo = (): void => {
    if (task.trim() === '') return;
    const newTodo: Todo = { id:
```

```

Date.now().toString(), text: task };
    setTodos([...todos, newTodo]);
    setTask('');
  };

  const removeTodo = (id: string): void => {
    setTodos(todos.filter((t) => t.id !== id));
  };

  return (
    <View style={styles.container}>
      <Text style={styles.title}>My To-Do
List</Text>
      <View style={styles.row}>
        <TextInput
          style={styles.input}
          placeholder="Add a task..."
          value={task}
          onChangeText={setTask}
        />
        <Pressable style={styles.addButton}
onPress={addTodo}>
          <Text style={styles.addButtonText}>+
</Text>
        </Pressable>
      </View>
      <FlatList
        data={todos}
        keyExtractor={({item}) => item.id}
        renderItem={({ item }) => (
          <Pressable style={styles.todoItem}
onPress={() => removeTodo(item.id)}>
            <Text>{item.text}</Text>
            <Text style=
{styles.remove}>X</Text>
          </Pressable>
        )}
      </FlatList>
    </View>
  );
}

```

```

    />
  </View>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, paddingTop: 60,
paddingHorizontal: 20 },
  title: { fontSize: 24, fontWeight: 'bold',
marginBottom: 20 },
  row: { flexDirection: 'row', marginBottom: 20
},
  input: { flex: 1, borderWidth: 1, borderColor:
'#ccc', borderRadius: 8, padding: 10,
marginRight: 10 },
  addButton: { backgroundColor: '#2f95dc',
borderRadius: 8, paddingHorizontal: 18,
justifyContent: 'center' },
  addButtonText: { color: '#fff', fontSize: 20
},
  todoItem: { flexDirection: 'row',
justifyContent: 'space-between', padding: 15,
borderBottomWidth: 1, borderBottomColor: '#eee'
},
  remove: { color: '#e74c3c' },
});

```

Every piece of data here has a clear, checked shape: `task` is always a `string`, `todos` is always an array of `Todo` objects, and both functions declare what they return (`void`, meaning "nothing"). Tap `+` to add a task, tap a task to remove it.

Common Beginner Questions

"Do I have to type everything?"

No. Let TypeScript infer types whenever it can (like `useState(0)`), and only add explicit types when starting from an empty/ambiguous value, defining props, or describing data shapes like `Todo`.

"What if I don't know the type yet?"

Avoid reaching for `any` (which turns off type checking entirely) — it's tempting as a beginner but defeats the purpose. If you're unsure, hover over a variable in VS Code and it will usually tell you what TypeScript already inferred.

"Why did my app still work with a type error showing?"

TypeScript errors show in your editor and fail your build, but Expo's dev server may still run the JavaScript underneath in development mode. Don't ignore red squiggly lines — fix them before shipping.

Next Steps

- Learn to type navigation between screens (React Navigation) — this is usually the next hurdle after basic components.
- Try typing a custom function that fetches data from an API and returns a typed result.
- Explore VS Code's "Go to Definition" (right-click a type name) — it's the fastest way to understand a library's types without reading external docs.